# Treavue

Personalized Travel Itinerary Planner

Project README • Repository Documentation

**Treavue** is a travel-planning project focused on helping users organize trip ideas and build personalized itineraries through a clean web interface. The intended project architecture uses a React frontend, a FastAPI backend, and Supabase integration. Always check the actual source code and environment configuration before describing a feature as implemented.

## Documentation note

This PDF is a readable README-style document based on the Treavue project context shared in this conversation. Exact API routes, deployed URLs, authentication behavior, and environment variable names should be verified against the latest working repository before public release.

# Table of Contents

1. Project Overview
2. Core Capabilities
3. System Architecture & Data Flow
4. Technology Stack
5. Project Directory Structure
6. Installation & Setup (Windows)
7. Environment Configuration
8. Running the Application
9. Testing & Validation
10. API Reference
11. Security, Limitations & Responsible Use
12. Future Enhancements

# 1. Project Overview

Treavue is a personalized travel itinerary planner designed to make trip planning more organized. The frontend provides the user-facing experience, while the planned backend layer handles application logic and connects to Supabase for supported data operations.

**Project goals**

• Provide a simple, responsive interface for planning trips.

• Organize itinerary details in one place.

• Support preference-aware itinerary planning as implemented by the current codebase.

• Use a structured frontend/backend design that can be extended with authentication and persistent storage.

# 2. Core Capabilities

## 2.1 Travel Planning Interface

The React interface is the primary place for users to interact with Treavue. Confirm each screen and workflow against the current frontend before listing it as a completed feature in a public portfolio.

## 2.2 Itinerary Organization

The project is intended to help users organize trip information and itinerary items. Persistence, edit/delete behavior, and synchronization should be described only to the extent that they work in the current build.

## 2.3 Preference-Based Planning

Preference-aware itinerary generation has been discussed as part of the project work. The quality of suggestions depends on the implemented logic and any external services used; recommendations should be reviewed by the traveler.

## 2.4 Data Integration

Supabase is part of the intended architecture for storing and retrieving trip data. Confirm that the latest backend is configured with the correct project URL, keys, policies, and user-ownership rules.

## 2.5 Responsive Experience

The frontend should be tested at desktop and mobile widths. Do not claim accessibility or browser compatibility certification without a dedicated validation pass.

# 3. System Architecture & Data Flow

```
Traveler
   |
   v
React Frontend (Vite)
   |  HTTP requests
   v
FastAPI Backend
   |  validated application logic
   v
Supabase
   |  persistent trip data (when configured)
   v
Frontend renders trip / itinerary results
```

Expected responsibility boundaries:

• **Frontend:** UI, user input, itinerary display, loading and error states.

• **Backend:** request validation, planning logic, API responses, and controlled access to data.

• **Supabase:** persistent data storage and related access policies, if enabled in the deployed environment.

The diagram is a high-level target architecture. Validate the current repository before treating every connection as active.

# 4. Technology Stack

| Layer | Technology | Role |
|---|---|---|
| Frontend | React, TypeScript, Vite | Web interface and development server |
| Styling | Project CSS / UI components | Layout, visual design, responsive behavior |
| Backend (intended) | Python, FastAPI | API and server-side planning logic |
| Data layer (intended) | Supabase | Persistent application data |
| Development tools | Node.js, pnpm/npm, Python | Dependency installation and local development |

Use the exact package versions from the lockfiles and requirements files as the source of truth. This table describes the project stack at a high level rather than asserting specific installed versions.

# 5. Project Directory Structure

```
Treavue-Travel-Planner/
■■■ artifacts/
■   ■■■ treavue/          # React/Vite frontend workspace
■       ■■■ src/          # Frontend source (verify exact files)
■       ■■■ public/       # Static assets (if present)
■       ■■■ package.json  # Frontend scripts and dependencies
■■■ backend/              # Intended FastAPI backend (verify in current repo)
■■■ README.md             # Project documentation
■■■ .env.example          # Recommended sample configuration
■■■ .gitignore            # Excludes local secrets and generated files
```

This is a simplified overview, not a guaranteed full tree. Update it to match the actual repository before publishing.

# 6. Installation & Setup (Windows)

## Prerequisites

• Git

• Node.js and a package manager compatible with the lockfile (pnpm or npm)

• Python version supported by backend dependencies

• Supabase project credentials if the data integration is enabled

### Step 1 — Clone the repository

```
git clone https://github.com/Mantukrv79/Treavue-Travel-Planner.git
cd Treavue-Travel-Planner
```

### Step 2 — Install frontend dependencies

Open the frontend workspace that contains the package.json. In the discussed repository layout, that is the artifacts/treavue folder:

```
cd artifacts/treavue
pnpm install
pnpm dev
```

If the repository uses npm instead, follow its lockfile and scripts (for example, npm install and npm run dev). Avoid mixing package managers in the same workspace.

### Step 3 — Set up the backend

If the current repository contains the FastAPI backend, open a separate terminal at its folder, create a virtual environment, activate it, and install the dependencies from the backend's actual requirements file.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

The requirements filename and startup module may differ. Check the backend directory and its README or application entry point before running a command.

## 7. Environment Configuration

Create a local environment file from the repository's example file, if provided. Use the exact variable names expected by the code; common names may include a Supabase project URL and a server-side key, but these names must be confirmed from the implementation.

```
# Example placeholders only — verify names in the source code
SUPABASE_URL=your_supabase_project_url
SUPABASE_KEY=your_key_for_the_intended_runtime
```

**Security:** Never commit real secrets, service-role keys, access tokens, or private credentials. A service-role key must never be exposed to browser code. Use least-privilege access policies and keep private keys on the server.

## 8. Running the Application

### Frontend

Run the frontend development command from the directory containing its package.json. The previously discussed Vite setup used:

```
pnpm dev
# Common local Vite address:
http://localhost:5173
```

### Backend

Run the FastAPI application using the entry point defined in the actual repository. A typical FastAPI project may be started with Uvicorn, but the module path and port must match your code.

```
# Example only; replace module:app with the actual entry point
uvicorn main:app --reload --port 8001
```

If Swagger/OpenAPI is enabled, FastAPI commonly exposes interactive API documentation at /docs. Verify the actual local backend URL and routes before sharing them.

# 9. Testing & Validation

Before publishing the repository or recording a demo, validate the project end-to-end:

• Install dependencies from a clean checkout.

• Run the frontend build and TypeScript checks if configured.

• Run backend tests using the project's documented test command.

• Test itinerary creation, editing, deletion, and persistence if those operations are implemented.

• Check empty states, invalid inputs, API failures, and slow network responses.

• Verify Supabase policies and ensure one user cannot access another user's private trips.

Do not include a test count, coverage percentage, or performance claim unless it comes from a recent, reproducible test run.

# 10. API Reference

Document only routes present in the current FastAPI application. The exact route list was not fully confirmed from the README reference alone.

Recommended API documentation checklist:

• HTTP method and path

• Required request fields and validation rules

• Response schema and status codes

• Authentication requirements and ownership checks

• Example success and error responses

If FastAPI is configured with OpenAPI, use its /docs page as the source of truth for the available endpoints.

# 11. Security, Limitations & Responsible Use

## Authentication and ownership

Every private trip operation should verify the authenticated user and enforce ownership on the server. Hiding UI controls is not an authorization mechanism.

## Supabase policies

Configure Row Level Security and policies appropriate to the schema. Do not rely on a client-supplied user ID as proof of identity.

## External travel information

Travel times, opening hours, prices, weather, and availability can change. Show the source and retrieval time where applicable and ask users to verify details before travel.

## AI-generated recommendations

If AI is used, itineraries can be incomplete or inaccurate. Users should verify safety, visa, transport, and booking details independently.

## Secrets and deployment

Keep backend secrets out of source control and browser bundles. Configure production CORS, HTTPS, logging, and error handling before public deployment.

### Scope of this README

Some technical details remain to be checked against the latest working repository. Update this document after verifying the real routes, setup commands, and feature status.

## 12. Future Enhancements

- Complete and verify secure authentication and per-user trip ownership.
- Improve itinerary personalization with clear user-controlled preferences.
- Add robust validation, meaningful errors, and loading states.
- Add tests for API behavior and database access policies.
- Improve mobile responsiveness and accessibility.
- Add reliable integration tests for the deployed frontend and backend.

Treavue — Personalized Travel Itinerary Planner

Before publishing, confirm that this document matches the latest source code and remove any capability that is not implemented.